

Lesson 9: Fetch API

Intro to Programming — Code the Dream

Session Overview

By the end of this session, you will be able to:

- Explain what an API is and why developers use them
- Use `fetch()` to request data from a server
- Handle a server response using `.then()` and `.catch()`
- Write an `async/await` function to do the same thing
- Display fetched data on a web page using the DOM

Explore

What Is an API?

API stands for **Application Programming Interface**.

Think of it as a menu at a restaurant:

- The kitchen has data (the food)
- You send a request (your order)
- The kitchen sends back a response (your meal)

You do not need to know how the kitchen works — just how to order.

In web development, APIs let your code ask a server for data.

What Is JSON?

JSON stands for **JavaScript Object Notation**.

It is the most common format for API data.

```
{  
  "name": "intro-to-programming",  
  "language": "JavaScript",  
  "private": false  
}
```

- It looks like a JavaScript object, but it is a **string**
- We use `response.json()` to convert it into real JavaScript data

What Is `fetch()` ?

`fetch()` is a built-in browser function that sends a request to a server.

```
fetch("https://api.github.com/users/octocat/repos")
```

- The URL tells fetch **where to send the request**
- By default, fetch sends a **GET** request — asking for data
- `fetch()` returns a **Promise** — the response will arrive later

`fetch()` is available in every major browser. No install needed.

Promises and `.then()`

`fetch()` does not give you data right away. It gives you a **Promise**.

A Promise is a placeholder that says: **"I will give you a value when I'm ready."**

We use `.then()` to say: **"When the data arrives, do this."**

```
fetch("https://api.github.com/users/octocat/repos")  
  .then(response => response.json())  
  .then(data => console.log(data));
```

Each `.then()` receives the result of the step before it.

Handling Errors with `.catch()`

What happens if the request fails? Use `.catch()`.

```
fetch("https://api.github.com/users/octocat/repos")  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error("Something went wrong:", error));
```

`.catch()` runs if **any step in the chain** throws an error.

Always include `.catch()`. Network requests can fail for many reasons.

Checking `response.ok`

`.catch()` only handles **network failures** (no connection, DNS errors).

If the server responds with an error code (like 404), `.catch()` won't run.

```
fetch("https://api.github.com/users/octocat/repos")
  .then(response => {
    if (!response.ok) {
      throw new Error("Request failed: " + response.status);
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Check `response.ok` to catch server-side errors too.

Check for Understanding

What does `fetch()` return immediately when you call it?

- A) The data from the server
- B) A Promise
- C) A JSON string
- D) `undefined`

async and await

async/await is a cleaner way to write the same fetch logic.

```
async function fetchRepos() {  
  const response = await fetch(  
    "https://api.github.com/users/octocat/repos"  
  );  
  const data = await response.json();  
  console.log(data);  
}  
  
fetchRepos();
```

- `async` marks the function as asynchronous
- `await` pauses the function until the Promise resolves

try...catch with async/await

Use `try...catch` for error handling — it replaces `.catch()`.

```
async function fetchRepos() {
  try {
    const response = await fetch(
      "https://api.github.com/users/octocat/repos"
    );
    if (!response.ok) throw new Error("Request failed");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Error:", error);
  }
}
```

Check for Understanding

You write `await fetch(url)` inside an `async` function. What happens?

- A) JavaScript throws an error — `await` is not allowed inside functions
- B) The function pauses at that line until the server responds, then continues
- C) The rest of your page freezes while waiting for the response
- D) `fetch()` runs but the result is ignored

From Data to the Page

Once you have data, use the DOM to display it.

```
const projectList = document.querySelector("#projects ul");

data.forEach(function(repo) {
  const item = document.createElement("li");
  item.textContent = repo.name;
  projectList.appendChild(item);
});
```

This is loops + DOM — skills you already have!

Check for Understanding

Which line actually adds a new item to the visible page?

```
const item = document.createElement("li");  
item.textContent = repo.name;  
projectList.appendChild(item);
```

- A) Line 1 — `createElement` creates it on the page
- B) Line 2 — setting `textContent` makes it visible
- C) Line 3 — `appendChild` inserts it into the DOM
- D) None — you also need `document.write()`

Apply

Core Activity: Fetch Your GitHub Repos

We will build this together, step by step.

Open two things:

1. Your `index.js` file in your code editor
2. Your browser console (right-click anywhere → Inspect → Console)

We will test in the console first, then move the code into `index.js`.

Step 1 — Test in the Console

Paste this into your browser console:

```
fetch("https://api.github.com/users/octocat/repos")  
  .then(response => response.json())  
  .then(data => console.log(data));
```

What do you see?

- Try expanding one object in the console
- Find the `name` property — that is what we will display

Step 2 — Add Error Checking

Let's also check that the response was successful:

```
fetch("https://api.github.com/users/octocat/repos")
  .then(response => {
    if (!response.ok) {
      throw new Error("Request failed");
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error("Error:", error));
```

Try it: Change `octocat` to a fake username. What happens?

Step 3 — Display Repos on the Page

Now let's move the code into `index.js` and show the repos on your portfolio.

```
const projectSection = document.getElementById("projects");
const projectList = projectSection.querySelector("ul");

fetch("https://api.github.com/users/YOUR_USERNAME/repos")
  .then(response => response.json())
  .then(function(repositories) {
    for (let i = 0; i < repositories.length; i++) {
      const project = document.createElement("li");
      project.textContent = repositories[i].name;
      projectList.appendChild(project);
    }
  })
  .catch(error => console.error("Error:", error));
```

Step 4 — Check It Works

Save your file and refresh your portfolio in the browser.

You should see:

- Your repository names listed under the Projects heading
- No errors in the console

Zoom chat: How many repos showed up?

If you see fewer than expected — GitHub's API returns 30 repos by default. That is normal.

Extension Activity

Only if time allows — this is optional.

Extension — Fetch a Different API

Try fetching from a public API that does not require authentication.

```
fetch("https://jsonplaceholder.typicode.com/posts/1")  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error(error));
```

Challenge: Can you display the `title` of the post on the page?

Harder: Fetch all posts (`/posts`) and show the first 5 titles in a list.

Assignment Preview

This week you will add **live GitHub data** to your portfolio:

Step	Task
1	Fetch your repos from <code>api.github.com</code>
2	Parse the JSON response
3	Loop through repos and add each as a <code></code>
4	Handle errors with <code>.catch()</code>
5	Style the projects list in <code>index.css</code>

Also: review the **Portfolio Rubric** — the final project is coming up!

Tips for the Assignment

1. **Test in the console first** — make sure your fetch works before adding DOM code
2. **Check `response.ok`** — a 404 will not trigger `.catch()` on its own
3. **Use your exact GitHub username** — capitalization matters
4. **Read the error message** — the console tells you what went wrong
5. **Stuck on the for loop?** It is the same pattern from the Arrays lesson

Ask for hints, not answers:

"Can you give me 3 hints without showing me the code?"

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

How comfortable do you feel using `fetch()` to get data?

Before next session:

- Work through the **lesson materials** (Odin Project: JSON, APIs, Async/Await)
- **Fetch your GitHub repos** and display them on your portfolio
- Review the **Portfolio Rubric** — know what is required
- Post questions in **Slack** anytime

Resources

Resource	What it's for
Odin Project — JSON	Understanding the data format
Odin Project — Working with APIs	How HTTP requests work
Odin Project — Async and Await	async/await deep dive
Scrimba — Make Network Requests	Video: fetch in action
MDN — Using Fetch	Full reference with examples
jsonplaceholder.typicode.com	Free fake API for practice

Next week: Open API Project — choose your own API!

Great work today!

Your portfolio now pulls live data from the web.

Questions? Reach out on Slack anytime.